Lecture-X
Data Visualization
Lecture-X

Bar Graphs

When you have categorical data, you can represent it with a bar graph. A bar graph plots data with the help of bars, which represent value on the y-axis and category on the x-axis. Bar graphs use bars with varying heights to show the data which belongs to a specific category.

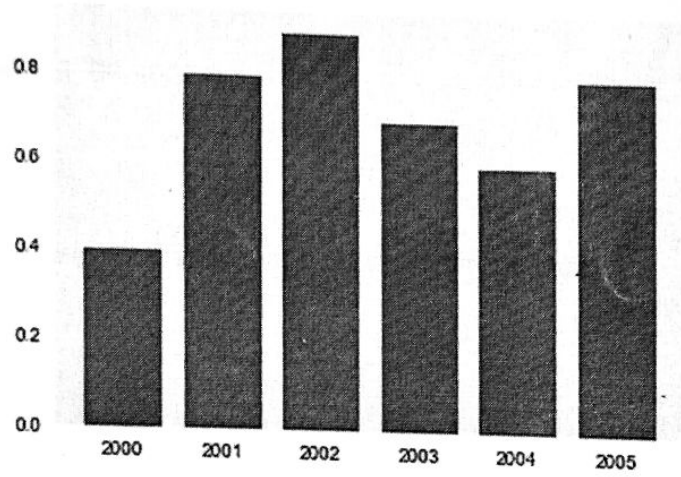
```
years = range(2000, 2006)
apples = [0.35, 0.6, 0.9, 0.8, 0.65, 0.8]
oranges = [0.4, 0.8, 0.9, 0.7, 0.6, 0.8]
```

```
plt.bar(years, oranges)

plt.xlabel('Year')
plt.ylabel('Yield (tons per hectare)')

plt.title("Crop Yields in Kanto")
```

```
<BarContainer object of 6 artists>
```



We can also stack bars on top of each other. Let's plot the data for apples and oranges.

```
plt.bar(years, apples)
plt.bar(years, oranges, bottom=apples)
```

```
<BarContainer object of 6 artists>
```

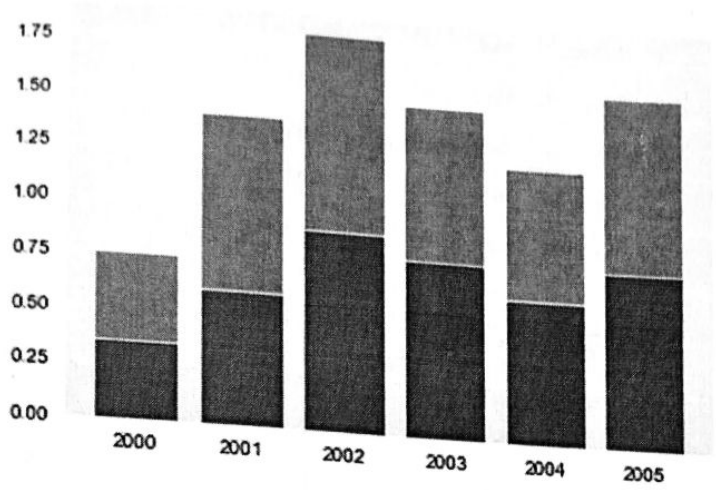


Figure 13: Plotting stacked bar graphs

Let's use the tips dataset in Seaborn next. The dataset consists of :

- Information about the gender.
- Time of day
- Total bill
- Tips given by customers visiting the restaurant for a week

```
tips_df = sns.load_dataset("tips")
tips_df
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4
...	...	...	...	...	...	...	...
239	29.03	5.92	Male	No	Sat	Dinner	3
240	27.18	2.00	Female	Yes	Sat	Dinner	2
241	22.67	2.00	Male	Yes	Sat	Dinner	2
242	17.82	1.75	Male	No	Sat	Dinner	2
243	18.78	3.00	Female	No	Thur	Dinner	2

244 rows × 7 columns

Figure 14: Iris Dataset

We can draw a bar chart to visualize how the average bill amount varies across different days of the week. We can do this by computing the day-wise averages and then using plt.bar. The Seaborn library also provides a barplot function that can automatically compute averages.

```
sns.barplot(x='day', y='total_bill', data=tips_df)
<matplotlib.axes._subplots.AxesSubplot at 0x2054afa7370>
```

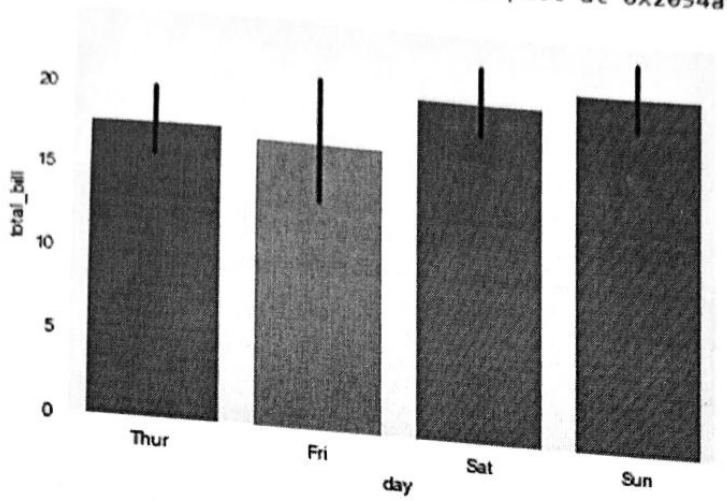


Figure 15: Plotting averages of each bar

If you want to compare bar plots side-by-side, you can use the hue argument. The comparison will be done based on the third feature specified in this argument.

```
sns.barplot(x='day', y='total_bill', hue='sex', data=tips_df)
<matplotlib.axes._subplots.AxesSubplot at 0x2054b046700>
```

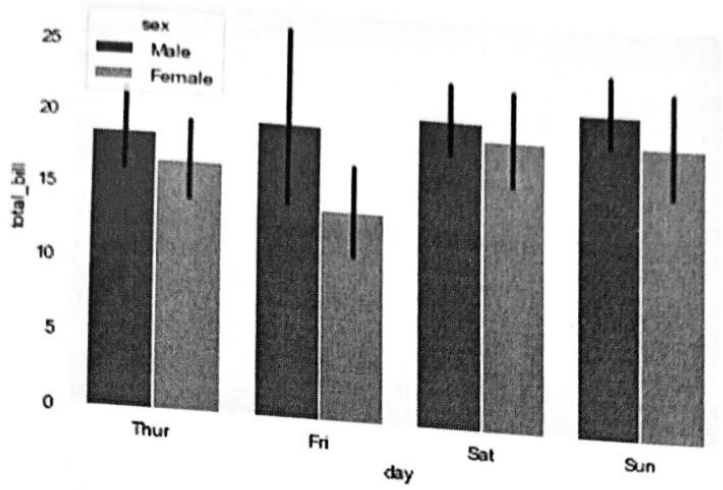


Figure 16: Plotting multiple bar graphs

You can make the bars horizontal by switching the axes.

```
sns.barplot(x='total_bill', y='day', hue='sex', data=tips_df)
<matplotlib.axes._subplots.AxesSubplot at 0x2054af1b250>
```

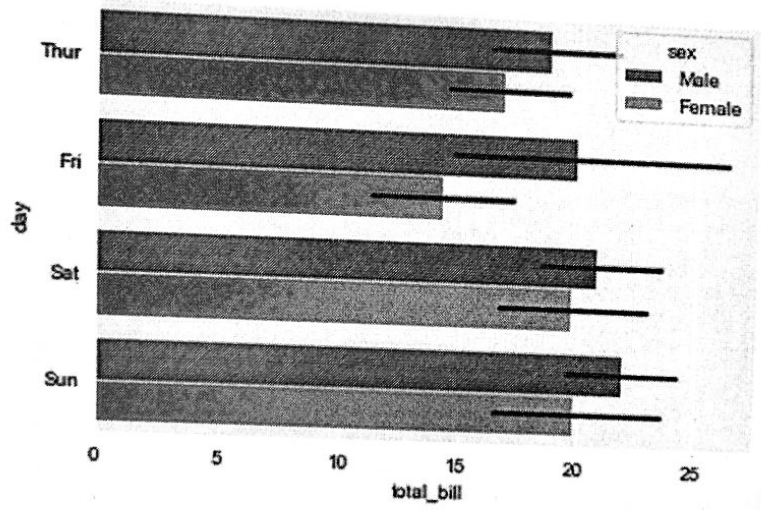


Figure 17: Plotting horizontal bar graphs

Histograms

A Histogram is a bar representation of data that varies over a range. It plots the height of the data belonging to a range along the y-axis and the range along the x-axis. Histograms are used to plot data over a range of values. They use a bar representation to show the data belonging to each range. Let's again use the 'Iris' data which contains information about flowers to plot histograms.

```
flowers_df = sns.load_dataset("iris")
flowers_df.sepal_width
```

```
0      3.5
1      3.0
2      3.2
3      3.1
4      3.6
      ...
145    3.0
146    2.5
147    3.0
148    3.4
149    3.0
Name: sepal_width, Length: 150, dtype: float64
```

Figure 18: Iris datase

Now, let's plot a histogram using the hist() function.

```
plt.title("Distribution of Sepal Width")
plt.hist(flowers_df.sepal_width)
```

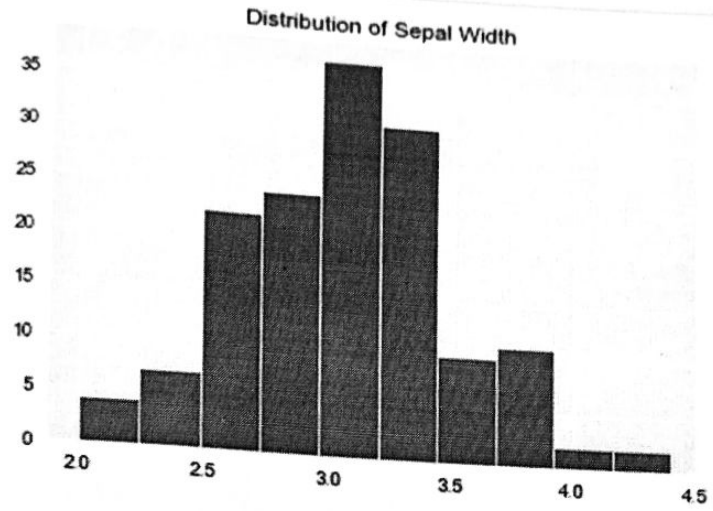


Figure 19: Plotting histograms

We can control the number or size of bins too.

```
# Specifying the number of bins
plt.hist(flowers_df.sepal_width, bins=5)
```

```
(array([11., 46., 68., 21.,  4.]),
 array([2.  , 2.48, 2.96, 3.44, 3.92, 4.4 ]),
 <a list of 5 Patch objects>)
```

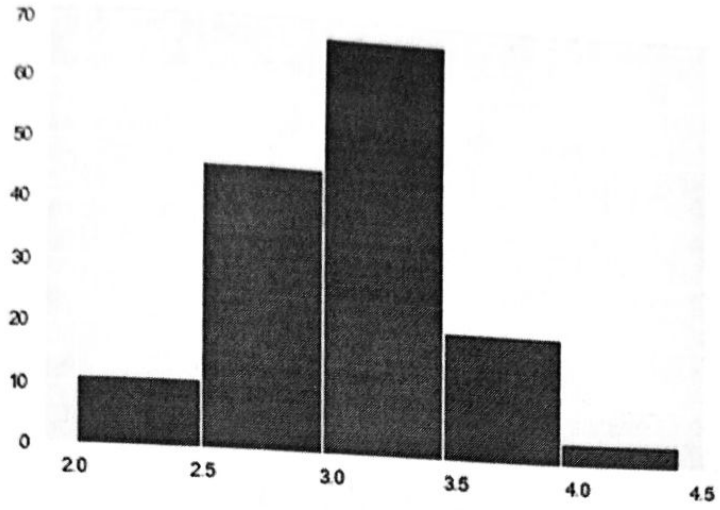


Figure 20: Changing number of bins

We can change the number and size of bins using numpy too.

```
import numpy as np

# Specifying the boundaries of each bin
plt.hist(flowers_df.sepal_width, bins=np.arange(2, 5, 0.25))
```

```
(array([ 4.,  7., 22., 24., 50., 18., 13.,  8.,  3.,  1.,  0.]),
 array([2.  , 2.25, 2.5 , 2.75, 3.  , 3.25, 3.5 , 3.75, 4.  , 4.25, 4.5 ,
        4.75]),
 <a list of 11 Patch objects>)
```

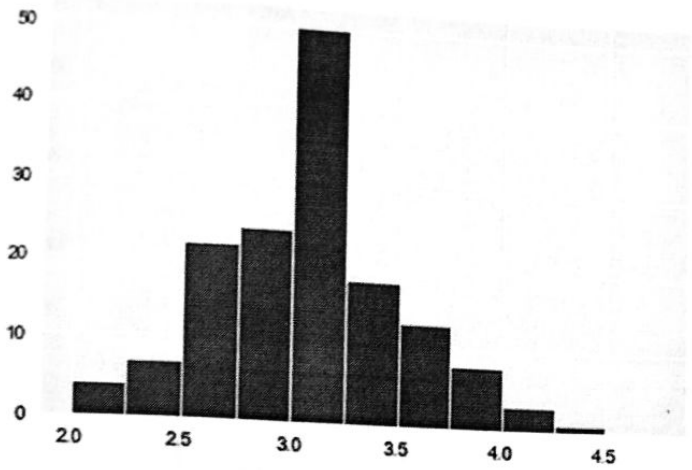


Figure 21: Changing number and size of bins

We can create bins of unequal size too.

```
# Bins of unequal sizes
plt.hist(flowers_df.sepal_width, bins=[1, 3, 4, 4.5])
```

```
(array([57., 89.,  4.]),
 array([1. , 3. , 4. , 4.5]),
 <a list of 3 Patch objects>)
```

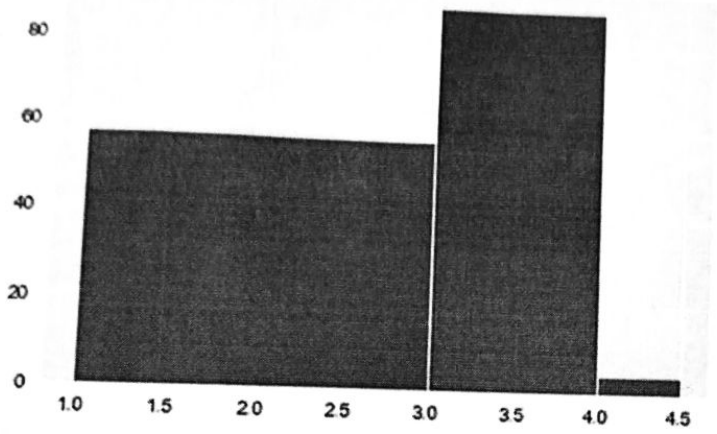


Figure 22: Bins of unequal size

Pie charts and stacked bar charts:

These visual show change in one or more quan-tities by plotting a series of data points over time and are frequently used with